

Seminar 2

week2(5- 9 October 2026)

please join Teams: s0p8jb7

Agenda

1. Introduction
2. Build Automation – Gradle & Maven
3. Version Control with Git
4. Further resources

Introduction

Modern software development relies on two categories of tooling that are independent of any specific programming language feature, yet essential to working on any real project: **build automation** and **version control**.

Build tools transform source code and its dependencies into a runnable, deployable artifact in a reproducible way.

Version control systems track and coordinate changes to that source code over time, particularly when multiple people work on it concurrently.

Build Automation

Why Build Tools?

Compiling a single Java file by hand with `javac` is simple, but real projects involve many source files, external libraries, resources and a packaging step. This does not scale manually and is not reproducible across machines.

A build tool automates the full pipeline:

- Compiling source code into bytecode
- Resolving and downloading **dependencies**
- Running **tests**
- Packaging everything into a single distributable artifact
- Producing a **reproducible build** – the same source always yields the same artifact

Gradle: Overview & Project Structure

Gradle is a build automation tool using a declarative **Groovy** or **Kotlin DSL** (build.gradle). It handles compilation, testing, dependency management and packaging.

File/Directory	Purpose
build.gradle	Main build configuration — dependencies, plugins, behaviour
settings.gradle	Declares project modules; required for multi-module projects
src/main/java	Application source code
src/main/resources	Non-code resources (e.g.: configuration files)
build/	Generated output — must not be version-controlled

Gradle: The Wrapper & Common Commands

The **Gradle Wrapper** (`gradlew`) is a script bundled with the project that downloads and uses a pinned Gradle version automatically, guaranteeing every contributor builds with the exact same version. Projects should be built with `./gradlew <task>`, not a system-wide install.

Command	Effect
<code>./gradlew build</code>	Compiles, tests and packages the project
<code>./gradlew assemble</code>	Packages the project without running tests
<code>./gradlew test</code>	Runs the test suite only
<code>./gradlew clean</code>	Deletes the build/ directory
<code>./gradlew bootJar</code>	(Spring Boot) Produces a fat / uber JAR

Gradle: Declaring Dependencies

Dependencies are declared in the `dependencies {}` block, using a configuration that determines when each is available:

```
dependencies {  
    implementation 'org.junit.jupiter: junit-jupiter:5.10.0'  
    testImplementation 'org.junit.jupiter: junit-jupiter-engine:5.10.0'  
}
```

implementation – available everywhere

testImplementation – available only for tests

api – available everywhere and exposed to consumers of this module

compileOnly – available only at compile time, not included at runtime

runtimeOnly – available only at runtime, not at compile time

annotationProcessor – used only for annotation processing during compilation

testCompileOnly – compile-time only, for tests

testRuntimeOnly – runtime only, for tests

Maven: Structure & Build Lifecycle

Maven uses a declarative **XML** configuration (**pom.xml**) and follows **convention over configuration**: little explicit setup is required as long as the project matches Maven's expected layout.

Build lifecycle

- a fixed sequence of phases
- running one runs every phase before it:

validate → **compile** → **test** → **package** → **verify** → **install** → **deploy**

File/Directory	Purpose
pom.xml	Build configuration — dependencies, plugins, metadata
src/main/java	Application source code
src/main/resources	Non-code resources
target/	Generated output — must not be version-controlled

Maven: Common Commands & Dependencies

Command	Effect
mvn compile	Compiles the project
mvn test	Compiles and runs tests
mvn package	Compiles, tests, and packages (JAR in target/)
mvn clean	Deletes the target/ directory
mvn install	Packages and installs to the local repository

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

no scope → defaults to **compile** (available everywhere)

scope=test → available only for tests

scope=provided → compile-time only, supplied at runtime

Gradle vs. Maven

GRADLE

Groovy/Kotlin DSL (code)

Flexible task graph

Configurable, imperative-capable

Faster on large/incremental builds

Common in Android & modern Java/Kotlin

MAVEN

XML configuration

Fixed lifecycle of phases

Convention over configuration

Simpler, typically slower on large builds

Common in enterprise & legacy Java

Dependency Resolution

A **dependency** is an external library a project needs to compile or run.

Once declared, the build tool:

- Resolves it against a remote repository (typically **Maven Central**)
- Downloads it automatically – no manual downloading
- Caches it locally for subsequent builds
- Includes it on the classpath, and bundles it into a fat/uber JAR if one is produced

Transitive dependencies – a dependency may itself depend on other libraries, resolved automatically. When two required versions of the same library conflict, a **version conflict** occurs.

Common Pitfalls

Committing build output – build/ or target/ is generated and belongs in .gitignore, never in version control.

Bypassing the wrapper – using a system-installed Gradle/Maven instead of the project's wrapper can produce builds that behave differently for different contributors.

Unpinned dependency versions – declaring a dependency without a specific version can silently change what is built over time.

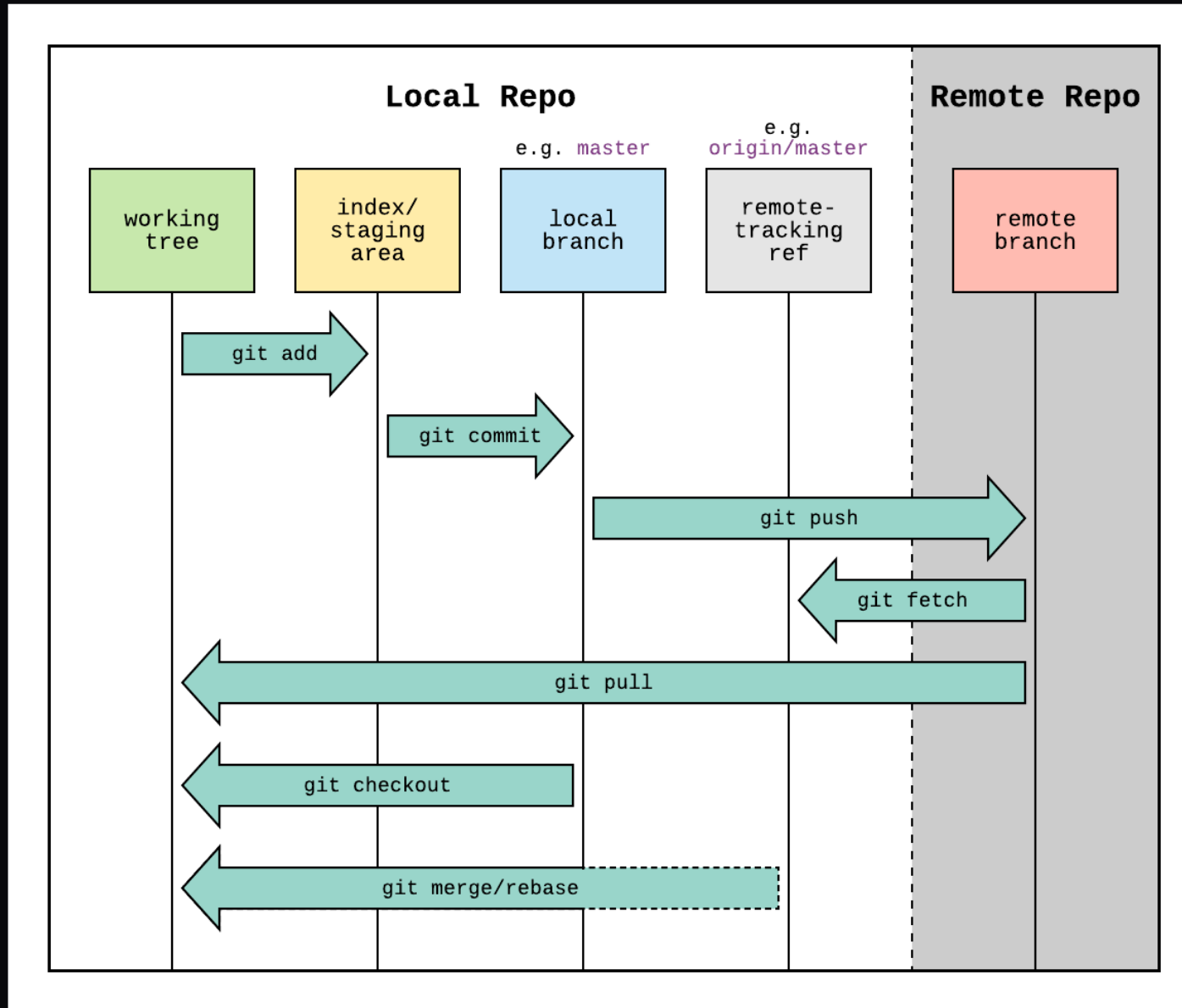
Version Control with Git

The Problem Version Control Solves

Version control (also **source control**) is the practice of tracking and managing changes to source code over time – recording history, enabling concurrent collaboration and providing safe ways to combine, compare and revert changes.

Git is a free, open-source, **distributed** version control system: every contributor holds a complete copy of the project's history locally, rather than relying on a single central server. Most operations (viewing history, committing, branching) happen entirely locally.

Git's Data Model



Core Commands: Snapshotting

Command	Effect
git init	Initializes a new Git repository
git status	Shows the state of tracked/untracked/staged files
git add <file>	Moves changes into the staging area
git commit -m "..."	Records staged changes as a new commit
git commit --amend	Modifies the most recent commit
git reset	Moves committed changes back out of history

git reset

--soft (keep staged)

--mixed (keep in working tree, default)

--hard (discard – destructive)

Branching, Merging & Conflicts

Command	Effect
git branch <name>	Creates a branch (no args: lists branches)
git checkout <branch>	Switches to the given branch
git checkout -b <branch>	Creates and switches in one step
git merge <branch>	Incorporates changes from the named branch
git log --oneline	Compact, one-line-per-commit history

Merge conflicts occur when the same part of a file was changed differently on both branches. Git marks the conflicting sections and pauses the merge until they are resolved manually and committed.

Remote Operations

Command	Effect
git fetch	Downloads remote changes without merging
git pull	Fetches and merges remote changes
git pull --rebase	Fetches and reapplies local commits on top
git push	Uploads committed changes to the remote
git push -u	Pushes and sets the upstream tracking branch
git push -f	Overwrites remote history — use with caution

Rewriting History

`git cherry-pick` — applies a single specific commit from one branch onto the current branch

`git rebase` — moves the current branch's commits on top of the named branch, producing linear history

`git revert` — creates a **new** commit undoing a specified commit, without altering existing history

Merge vs. Rebase

Merge

Creates a new merge commit

Non-destructive — original commits unchanged

Can clutter history with merge commits

Rebase

Reapplies commits, producing linear history

Rewrites history — creates new commits

Cleaner history if done carefully

The Golden Rule of Rebasing:

Never rebase a branch that others are working from — in particular, never rebase a shared or public branch such as **main/master**

Rebasing is safe on local, unshared feature branches.

Revert vs. Reset

RESET

Removes commits from history

Rejected by the remote if already pushed (unless forced)

Best for local, unpushed mistakes

REVERT

Preserves history, adds a new undo commit

Safe to push normally – no conflict

Best for undoing already-shared changes

.gitignore

Not every file should be tracked. Files that are system-dependent, generated, or contain secrets should be excluded via a **.gitignore** file:

```
# IDE files
.idea/
*.iml

# Build output
build/
target/

# Compiled files
*.class
```

Tooling: CLI & IntelliJ IDEA

Every Git operation covered in this seminar is available both from the command line and through IntelliJ IDEA's built-in Git integration (the **Git** menu and the **Commit** tool window).

IntelliJ's interface is a graphical wrapper around the same underlying commands – understanding the commands makes the IDE's menus and dialogs self-explanatory.

Further Resources

Gradle - https://docs.gradle.org/current/userguide/getting_started.html

Maven - <https://maven.apache.org/guides/getting-started/>

Git - <https://git-scm.com/book/en/v2>

Thank you!

Seminar material has been prepared by Tudor Jinga and Diana Timiș